PROGRAMAÇÃO CONCORRENTE EM GO

FPPD — Unidade 2

16 de Março, 2026

ROTEIRO

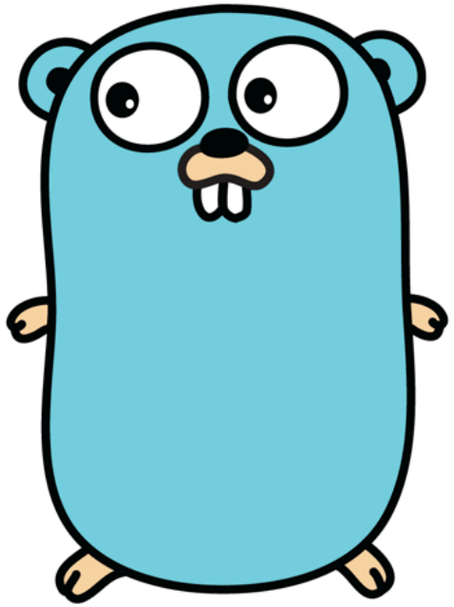
1. **A Linguagem Go**
   Origens, características e estrutura de um programa
2. **Goroutines**
   Criação, exemplos e comportamento não-determinístico
3. **Sincronização com WaitGroup**
   Esperando goroutines terminarem
4. **Exclusão Mútua**
   Mutex e proteção de seções críticas

A LINGUAGEM GO

Simples, eficiente e concorrente

Build simple, secure, scalable systems

- Criada pela **Google**, código aberto desde **2009**
- Projetada por **Robert Griesemer**, **Rob Pike** e **Ken Thompson**
- Linguagem **compilada** e **estaticamente tipada**
- Focada em **simplicidade**, **eficiência** e **concorrência**
- Garbage collector integrado
- Compilação extremamente rápida



Gopher — mascote oficial da linguagem Go

OS CRIADORES DO GO

KEN THOMPSON

- Co-criador do **UNIX**
- Criador da **linguagem B** (predecessora do C)
- Criador do **grep** e das **expressões regulares**
- Co-inventor do **UTF-8**
- **Prêmio Turing** (1983)

ROB PIKE

- Arquiteto do **Plan 9**, sucessor do UNIX
- Co-inventor do **UTF-8**
- Criou **Newsqueak** e **Limbo** (concorrência via CSP)
- Influência direta nas **goroutines e canais**

ROBERT GRIESEMER

- Motor **V8** do JavaScript (Chrome / Node.js)
- JVM **HotSpot** (Java)
- Linguagem **Sawzall** no Google
- Influência no **compilador e runtime** de Go

A simplicidade de Go e seu modelo de concorrência não são acidentais — vêm de **décadas de experiência** em sistemas operacionais, linguagens e compiladores.

QUEM USA GO?

Empresas de todos os setores usam Go em produção

Google	Microsoft	Meta	NETFLIX
Uber			CLOUDFLARE
twitch	mercado libre	AMERICAN EXPRESS	X
RIOT GAMES	ByteDance	salesforce	bitly

> Docker, Kubernetes e Terraform — ferramentas essenciais da infraestrutura moderna — são escritos em Go.

Fonte: go.dev

POR QUE GO PARA CONCORRÊNCIA?

GOROUTINES

- Threads leves gerenciadas pelo runtime de Go
- Criação com uma única palavra-chave: `go`
- Milhares simultâneas com baixo custo

CANAIS

- Comunicação nativa entre goroutines
- Evita compartilhamento direto de memória
- *(próximas aulas)*

DETECTOR DE CORRIDA

- Ferramenta integrada: `go run -race`
- Detecta data races em tempo de execução
- Essencial para depuração

> Go foi projetada desde o início pensando em concorrência — não é um recurso adicionado depois.

ESTRUTURA DE UM PROGRAMA GO

```go
package main

import "fmt"

func main() {
    fmt.Println("Olá, FPPD!")
}
```

- `package main` — define o pacote principal (executável)
- `import "fmt"` — importa o pacote de formatação de E/S
- `func main()` — ponto de entrada do programa

go run hello.go | go build hello.go | ./hello

GOROUTINES

Concorrência leve em Go

O QUE SÃO GOROUTINES?

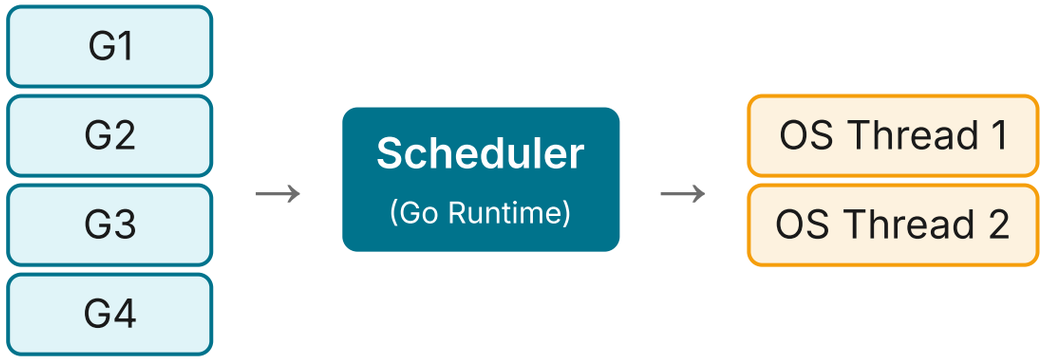
> Uma goroutine é uma **função leve** executada concorrentemente com outras goroutines.

- **Não** é uma thread tradicional do sistema operacional
- Implementação própria: **user-level threads** (green threads)
- Pilha inicial pequena (~2 KB) e **expansível dinamicamente**
- É possível executar **milhares ou milhões** simultaneamente
- Gerenciadas pelo **runtime** de Go, não pelo SO

> A função `main()` é, ela própria, uma goroutine — a **goroutine principal**.

GOROUTINES VS THREADS DO SO

Aspecto	Threads do SO	Goroutines
Pilha inicial	~1 MB (fixa)	~2 KB (expansível)
Criação	Custosa (chamada de sistema)	Barata (runtime de Go)
Escalonamento	Pelo kernel do SO	Pelo runtime de Go (cooperativo)
Quantidade prática	Centenas a poucos milhares	Centenas de milhares ou mais
Troca de contexto	Cara (modo kernel)	Barata (modo usuário)



Modelo M:N — múltiplas goroutines multiplexadas em poucas threads do SO

CRIAÇÃO DE GOROUTINES

Basta adicionar a palavra-chave go antes de uma chamada de função.

CHAMADA NORMAL

```go
func minhaFuncao() {
    fmt.Println("Executando...")
}

// Chamada sequencial (bloqueia)
minhaFuncao()
fmt.Println("Próxima linha")
```

GOROUTINE

```go
func minhaFuncao() {
    fmt.Println("Executando...")
}

// Inicia uma goroutine (não bloqueia)
go minhaFuncao()
fmt.Println("Próxima linha")
```

A chamada com go retorna **imediatamente** — a função executa em paralelo.

EXEMPLO: SEQUENCIAL VS CONCORRENTE

SEQUENCIAL

```go
func diga(msg string) {
    for i := 0; i < 3; i++ {
        fmt.Println(msg)
        time.Sleep(500 * time.Millisecond)
    }
}

func main() {
    diga("Olá!")  // bloqueia
    diga("Mundo") // só após terminar
}
```

CONCORRENTE

```go
func diga(msg string) {
    for i := 0; i < 3; i++ {
        fmt.Println(msg)
        time.Sleep(500 * time.Millisecond)
    }
}

func main() {
    go diga("Olá!") // goroutine
    diga("Mundo")   // executa na main
}
```

A única diferença é go antes da chamada. Mas o comportamento muda completamente.

SAÍDA: SEQUENCIAL VS CONCORRENTE

SEQUENCIAL

```
$ go run sequencial.go
Olá!
Olá!
Olá!
Mundo
Mundo
Mundo
```

Saída sempre na mesma ordem.

CONCORRENTE

```
$ go run concorrente.go
Mundo
Olá!
Olá!
Mundo
Mundo
Olá! ← pode não aparecer!
```

Ordem varia entre execuções.

A goroutine `diga("Olá!")` pode não terminar antes de `main()` — e quando `main()` retorna, todas as goroutines são interrompidas.

CONCORRÊNCIA ≠ ORDEM

> A **ordem de execução** das goroutines depende do escalonador do runtime — não do programador.

- O resultado de um programa concorrente pode **variar entre execuções**
- Isso é chamado de **não-determinismo** — comportamento esperado, não um bug
- A mesma entrada pode produzir saídas em **ordens diferentes**
- Concorrência introduz um novo desafio: como **coordenar** a execução?

CONDIÇÃO DE PARADA

Em Go, quando `main()` termina, **todas as goroutines são imediatamente interrompidas** — mesmo que não tenham terminado.

O PROBLEMA

- A goroutine pode ou não executar antes de `main()` retornar
- O programa termina sem aviso
- Trabalho pode ser **perdido silenciosamente**

SOLUÇÕES

- `time.Sleep()` — apenas para testes simples
- `sync.WaitGroup` — barreira de sincronização
- **Canais** — sincronização via comunicação *(próximas aulas)*

Nunca usar `time.Sleep()` em código de produção para sincronizar goroutines — o tempo pode variar entre máquinas e execuções.

ESPERANDO GOROUTINES

sync.WaitGroup

SYNC.WAITGROUP

`WaitGroup` é uma **barreira de sincronização** que espera pela conclusão de um conjunto de goroutines.

WG.ADD(N)

Incrementa o contador em `n` goroutines que devem ser aguardadas

WG.DONE()

Decrementa o contador em 1. Chamada quando a goroutine termina

WG.WAIT()

Bloqueia até que o contador chegue a **zero**

Add(n) → inicia goroutines → Done() ×n → Wait() desbloqueia

EXEMPLO COM WAITGROUP

```go
import (
    "fmt"
    "sync"
)

func tarefa(id int, wg *sync.WaitGroup) {
    defer wg.Done() // decrementa ao final
    fmt.Printf("Goroutine %d executando\n", id)
}

func main() {
    var wg sync.WaitGroup
    for i := 1; i <= 3; i++ {
        wg.Add(1)          // incrementa antes de lançar
        go tarefa(i, &wg)  // passa ponteiro do WaitGroup
    }
    wg.Wait() // bloqueia até todas terminarem
    fmt.Println("Todas as goroutines finalizaram!")
}
```

GOROUTINES NA PRÁTICA

Argumentos, funções anônimas e padrões comuns

PASSANDO ARGUMENTOS PARA GOROUTINES

Como uma goroutine é uma chamada de função, basta passar os argumentos normalmente.

```go
func imprimir(valor int) {
    fmt.Printf("Valor: %d\n", valor)
}

func main() {
    for i := 0; i < 5; i++ {
        go imprimir(i) // cada goroutine recebe uma cópia de i
    }
    time.Sleep(time.Second)
}
```

Atenção: os argumentos são avaliados no momento da chamada `go`. Cada goroutine recebe uma **cópia** do valor — não uma referência à variável do loop.

GOROUTINES ANÔNIMAS

`go func() { ... }()` cria e executa uma **função anônima** como goroutine — ideal para tarefas curtas.

SEM ARGUMENTOS

```go
func main() {
    go func() {
        fmt.Println("Paralelo!")
    }()  // () executa imediatamente
    fmt.Println("Main continua...")
    time.Sleep(100 * time.Millisecond)
}
```

COM ARGUMENTOS

```go
for i := 0; i < 5; i++ {
    go func(n int) {
        fmt.Printf("Goroutine %d\n", n)
    }(i) // passa i como argumento
}
```

EXEMPLO: MÚLTIPLAS GOROUTINES

```go
)

func worker(id int, tarefa string, wg *sync.WaitGroup) {
    defer wg.Done()
    fmt.Printf("Worker %d iniciou: %s\n", id, tarefa)
    time.Sleep(time.Duration(id) * 500 * time.Millisecond)
    fmt.Printf("Worker %d concluiu: %s\n", id, tarefa)
}

func main() {
    var wg sync.WaitGroup
    tarefas := []string{"compilar", "testar", "empacotar"}
    for i, t := range tarefas {
        wg.Add(1)
        go worker(i+1, t, &wg)
    }
    wg.Wait()
    fmt.Println("Todas as tarefas concluídas!")
}
```

> Cada worker leva um tempo diferente — a ordem de conclusão varia entre execuções.

EXCLUSÃO MÚTUA

Protegendo seções críticas com Mutex

O PROBLEMA: CONDIÇÃO DE CORRIDA

```go
var saldo int = 100

func sacar(valor int, wg *sync.WaitGroup) {
    defer wg.Done()
    if saldo >= valor {
        saldo -= valor // SEÇÃO CRÍTICA
    }
}

// na main:
wg.Add(2)
go sacar(80, &wg)
go sacar(80, &wg)
wg.Wait()
fmt.Println("Saldo:", saldo)
```

Esperado: apenas 1 saque (saldo = 20)
Real: ambos sacam! (saldo = −60)

As duas goroutines leem `saldo >= 80` como verdadeiro **antes** de qualquer uma subtrair — ambas passam pela verificação.

SYNC.MUTEX

`Mutex` (Mutual Exclusion) garante que apenas **uma goroutine por vez** acesse a seção crítica.

```go
var (
    saldo int = 100
    mu    sync.Mutex
)

func sacar(valor int, wg *sync.WaitGroup) {
    defer wg.Done()
    mu.Lock()          // bloqueia acesso
    if saldo >= valor {
        saldo -= valor // seção crítica protegida
    }
    mu.Unlock()        // libera acesso
}
```

`mu.Lock()`
Bloqueia — apenas uma goroutine entra

`mu.Unlock()`
Libera — outra goroutine pode entrar

SYNC.RWMUTEX

Variante que permite **múltiplas leituras simultâneas**, mas apenas **uma escrita por vez**.

Operação	Método	Comportamento
Leitura	`RLock()` / `RUnlock()`	Permite múltiplas leituras; bloqueia se há escrita
Escrita	`Lock()` / `Unlock()`	Exclusivo — bloqueia leituras e escritas

```go
var rwmu sync.RWMutex
func leitura() {
    rwmu.RLock()
    defer rwmu.RUnlock()
    // leitura segura — outras goroutines também podem ler
}
func escrita() {
    rwmu.Lock()
    defer rwmu.Unlock()
    // escrita exclusiva — ninguém mais acessa
}
```

DETECTOR DE CORRIDA

Go possui um **detector de data race integrado** que identifica acessos concorrentes não protegidos.

```
go run -race programa.go
```

```
$ go run -race contador.go
==================
WARNING: DATA RACE
Read at 0x00c0000b4010 by goroutine 7:
  main.incrementar()
    contador.go:13 +0x6e
Previous write at 0x00c0000b4010 by goroutine 8:
  main.incrementar()
    contador.go:13 +0x80
==================
Found 1 data race(s)
```

Sempre teste seus programas concorrentes com `-race` durante o desenvolvimento.

EXERCÍCIO PROPOSTO

> Crie um programa em Go com **duas goroutines** disparadas a partir da `main()`.

1. Uma goroutine deve imprimir números de **1 a 10** a cada 1 segundo
2. A outra goroutine deve imprimir números de **10 a 1** a cada 1 segundo
3. A `main()` deve esperar até que as duas goroutines terminem
4. Utilize **`sync.WaitGroup`** para fazer a `main()` esperar

RESUMO

- **Go** foi projetada para concorrência — simples, eficiente e compilada
- **Goroutines** são funções leves executadas concorrentemente com `go`
- O modelo **M:N** multiplexa goroutines em poucas threads do SO
- **WaitGroup** permite esperar goroutines terminarem corretamente
- **Mutex** protege seções críticas contra condições de corrida
- Sempre usar `go run -race` para detectar data races